

Sequential Evaluation and Generation Framework for Combinatorial Recommender System

Fan Wang*
Baidu Inc.
Shenzhen, China
wang.fan@baidu.com

Xiaomin Fang*
Baidu Inc.
Shenzhen, China
fangxiaomin01@baidu.com

Lihang Liu
Baidu Inc.
Shenzhen, China
liulihang@baidu.com

Yaxue Chen
Baidu Inc.
Shenzhen, China
chenyaxue@baidu.com

Jiucheng Tao
Baidu Inc.
Shenzhen, China
taojiucheng@baidu.com

Zhiming Peng
Baidu Inc.
Beijing, China
pengzhiming01@baidu.com

Cihang Jin
Baidu Inc.
Beijing, China
jincihang@baidu.com

Hao Tian
Baidu Research
Sunnyvale, United States
tianhao@baidu.com

ABSTRACT

In the combinatorial recommender systems, multiple items are fed to the user at one time in the result page, where the correlations among the items have impact on the user behavior. In this work, we model the combinatorial recommendation as the problem of generating a sequence(ordered list) of items from a candidate set, with the target of maximizing the expected overall utility(e.g. total clicks) of the sequence. Toward solving this problem, we propose the Evaluation-Generation framework. On the one hand of this framework, an evaluation model is trained to evaluate the expected overall utility, by fully considering the user, item information and the correlations among the co-exposed items. On the other hand, generation policies based on heuristic searching or reinforcement learning are devised to generate potential high-quality sequences, from which the evaluation model select one to expose. We propose effective model architectures and learning metrics under this framework. We also offer series of offline tests to thoroughly investigate the performance of the proposed framework, as supplements to the online experiments. Our results show obvious increase in performance compared with the previous solutions.

KEYWORDS

Recommender System, Intra-list Correlations, Diversified Ranking, Reinforcement Learning

1 INTRODUCTION

Recommender Systems(RS) attracts a lot of attention with the booming of information on the Internet. Typical RS algorithms include collaborative methods ([27], [21], [15]), content based methods and hybrid methods of the two([1]). Among the recent works, several topics have drawn more attentions: Context-Aware Recommendation([1]) seeks to utilize the information of scenes where the users are fed; Time-Aware RS([4]) focuses on the evolution of the user interest;

Diversified ranking([33]) tries to address the correlation among co-exposed items, i.e. the intra-list correlations.

It's important to consider the intra-list correlations in many realistic RS in order for better user experience. However, the research on intra-list correlations usually collapses to diversity ([33]) in many works. Yet we think that the investigation in intra-list correlation is far from enough in several aspects: Firstly, the evaluation of diversity misses a gold standard. Though there have been various metrics such as **Coverage** ([40]), **Intra-List Similarity** ([28]) etc, those evaluation metrics are typically subjective, and not directly related to the true user experience. Secondly, strong propositions have been imposed on the formulation of the correlation in the previous work. Algorithms such as determinant point process ([33]) and sub-modular ranking rely on handcrafted kernels or functions, which cannot effectively capture all possible correlation forms. Thirdly, the traditional step-wise greedy ranking method typically neglects the loss of the local optimum. If we consider the sub-modular ranking, the lower bound of the ratio of the overall utility comparing the greedy choice to the global optimum is $(1 - 1/e)$ ([35]). Yet, the loss of the local optimum is totally unclear when going beyond the sub-modularity hypothesis.

In this paper, we propose to optimize the overall utility of a sequence, with the preposition that the diversity and the other intra-list correlations need to be responsible for this utility. To do so, on the one hand, we build neural architectures to encode the list to predict the utility of the list. We call this part the **Evaluator**. On the other hand, we try to build ranking policies to generate recommendation lists from a candidate set of items, which we call the **Generator**. We use sequence decoder as the ranking policy, targeting at generating a sequence(a recommendation list) with as high overall utility as possible. This target can be typically realized by heuristic searching or reinforcement learning. In addition, we use the Generator to generate multiple potential high-quality lists, from which the Evaluator further selects the superior one to achieve high performance.

*Both authors contributed equally to this research.

Offline evaluation of combinatorial RS is challenging, as static data can not be used to evaluate such system. Previous work either use subject metrics([12],[40]) or simulators([35]), while the simulator itself is typically not validated. In this work, we adopt progressive evaluation steps to validate the Evaluator and the Generator respectively. The validity of the proposed framework is supported by both offline analysis and online experiments.

There are several contributions in this work:

- We provide a thorough investigation of intra-list correlations in a realistic recommender system. We propose model architectures that capture those correlations in more effective way compared with the previous methods.
- We propose practical offline evaluation metrics for combinatorial RS, which is consistent with the online experiments.
- The proposed recommender frameworks is fully launched in an online system with hundreds of millions of daily active users.

2 RELATED WORKS

Diversity has been frequently investigated in the area of intra-list correlations. Recent works on diversified ranking include the submodularity ([2],[34]), graph based methods ([39]), and Determinantal Point Process (DPP) ([22], [23], [33]). The diversified ranking with predefined submodular functions typically supposes that diversity are homogeneous on different topics, and independent of the user. DPP and submodular ranking also suppose that co-exposed items always have a negative impact on the possible click of the others. In contrast to those propositions, realistic RS show cases that violate those rules. Our statistics reveal that some related contents are prone to be clicked together. Except for combination, phenomenons relating the user feedback to display positions have also been widely studied, e.g., click models in Information Retrieval(IR) systems, such as the Cascade Click Model([9]) and the Dynamic Bayesian Network([5]). It is found that the position bias is not only related to the user's personal habit, but also related to the layout design etc([7]). Thus click models often need to be considered case-by-case. More complex phenomenon has also been discovered, such as **Serpentining**[36], which found that the users prefer discontinuous clicks when browsing the list. It is recommended that high quality items should be scattered over the entire list instead of clustered on the top positions. In contrast to those discoveries, few the previous works have studied the intra-list correlation and the position bias together.

Our framework draws ideas from model-based reinforcement learning([11]), where the transition probability and reward function is directly approximated through modeling the environment(here we have the Evaluator), and heuristic searching and other planning techniques are used to search for optimistic trajectories(just like the Generator). We also borrow ideas from applying model-free Reinforcement Learning for effective planning and searching. Various architectures and learning metrics has been proposed for similar problem, e.g. Vinyals et al. applied pointer network for universal combinatorial optimization(CO) problem([31]). Bello et al. further extended pointer-network by using policy gradients for learning([3]). Dai et al. applied Q-Learning to CO problems on graphs([19]). Those

works are focused on general optimization problems such as CO, while extension to RS requires further investigation.

Works on addressing different kinds of long-term rewards in RS have also been reported, too. Feng et al. applied policy gradient and Monte Carlo Tree Search(MCTS) to optimize the α -NDCG in diversified ranking for the global optimum([12]). Other works pursued long-term rewards in inter-list recommendations ([37], [38]). Though Zhao et al. also proposed treating a page of recommendation list as a whole([37]), the intra-list correlations are not sufficiently analyzed. Our work is fundamentally different in more thoroughly investigation of the intra-list correlations. Though inter-list correlations are also important, we regard it as an independent problem that is not studied in this paper.

3 METHODOLOGY

3.1 Problem Setup

We formulate the combinatorial RS as follow: The environment exposes the user profile u and a candidate set $\mathbf{c} = \{c_1, c_2, \dots, c_N\}$ to the RS, where N is the cardinality of the candidate set and c_i denotes the i -th item. The system picks a recommendation sequence $\mathbf{a} = [c_{a_1}, c_{a_2}, \dots, c_{a_K}]$ where $a_j \in [1, N]$ and $N \geq K$. $\mathbf{a}$ is exposed to the user, after which the user returns the feedback of $r(u, \mathbf{a})$. Furthermore, we denote $\mathbf{a}_j^- = [c_{a_1}, c_{a_2}, \dots, c_{a_{j-1}}]$ as the preceding recommendations above the j -th position, and $\mathbf{a}_j^+ = [c_{a_{j+1}}, c_{a_{j+2}}, \dots, c_{a_K}]$ as the recommendations that follows. We define the final objective as maximizing the expected overall utility of the sequence $\mathbf{a}$, which is written as $\mathbb{E}[r(u, \mathbf{a})]$, where we use $\mathbb{E}_X[\cdot]$ to represent the expectation over variable X , $\mathbb{E}[\cdot]$ simply represents the expectation over repeated experiments.

$$\mathbf{a}_{opt} = \operatorname{argmax}_{\mathbf{a}} \mathbb{E}[r(u, \mathbf{a})] \quad (1)$$

Typically, the overall utility $r(u, \mathbf{a})$ is the summation of utility(rewards) $r_j(u, \mathbf{a})$ (e.g., clicks) over all positions $j \in [1, K]$ in the list, i.e. $r(u, \mathbf{a}) = \sum_{j=1}^K r_j(u, \mathbf{a})$. We use $\mathbf{r}(u, \mathbf{a}) = [r_1, \dots, r_K]$ to denote the vector of rewards. Except for clicks, other feedbacks might be available in RS, such as dwelling time(either item-wise or dwelling time over the whole list) and request for subsequent recommendation. Some of the rewards can not be assigned to each item, while others can be. Our work is mainly focused on clicks, however, extension to other targets or multi-task learning is straight-forward.

3.2 Overview of the Evaluator-Generator Framework

We use $f_\theta(u, \mathbf{a})$ to represent the predicted overall utility from the Evaluator, and $p(\mathbf{a}|u, \mathbf{c}) = \Pi_\eta(\mathbf{a}; u, \mathbf{c})$ to represent the Generator. We want f_θ to approximate the ground truth $r(u, \mathbf{a})$, and we use Π_η to plan or search for better score. The target of the Evaluation-Generation framework is to solve equation. 2.

$$\begin{aligned} &\operatorname{argmax}_{\mathbf{a} \in \{\mathbf{a}_1, \dots, \mathbf{a}_n\}} f_\theta(u, \mathbf{a}), \\ &\text{with } f_\theta(u, \mathbf{a}) \rightarrow \mathbb{E}[r(u, \mathbf{a})] \text{ and } \mathbf{a}_1, \dots, \mathbf{a}_n \sim \Pi_\eta \end{aligned} \quad (2)$$

We use the formulation $X_\theta \rightarrow Y$ here to represent that "Approximate Y with model X_θ with respect to parameters θ ". Typically we reduce the mean square error (MSE) between X_θ and Y , or maximize

the log likelihood. A sketch of the framework is shown in fig. 1. On the one hand, we require the Evaluator to consider the sequence as a whole in order to fully capture position biases, diversity and other correlations etc before. We optimize the parameter θ by supervised learning to minimize the approximation error. On the other hand, we require the Generator to effectively plan the sequences that achieves higher overall utility. We argue that the solution of equation. 2 can effectively approximate the solution of equation. 1 by carefully selecting the model architecture and learning metrics. In the following part, we explain the model architectures and learning metrics for the Evaluator and Generator respectively.

3.3 The Evaluators

The Evaluator $f_\theta(u, \mathbf{a})$ encodes the whole sequence and approximates the overall utility. We argue that f_θ should have the following characteristics: f_θ needs to be sensitive to the order of the sequences, such that relation with position can be properly considered; f_θ should add as little artificial hypothesis as possible in order to account for all possible correlations. We propose to use sequence encoding structures, such as recursive neural structure and self-attention layers to enable sufficient interaction among the displayed items. We use MSE loss $L(\theta; f_\theta)$ for the optimization of θ , written as equation. 3.

$$\text{minimize } L(\theta; f_\theta) = \mathbb{E}_{u, \mathbf{a}} [(f_\theta(u, \mathbf{a}) - r(u, \mathbf{a}))^2] \quad (3)$$

While it is possible to directly predict the overall utility with $f_\theta(u, \mathbf{a})$, the item-wise feedback improves the performance by providing finer supervision. Take the user click as example, we require the evaluator to predict $\mathbb{E}[\mathbf{r}(u, \mathbf{a})]$ rather than $\mathbb{E}[r(u, \mathbf{a})]$. Thus we set the learning target as $\mathbf{f}_\theta = [f_{\theta,1}, \dots, f_{\theta,K}]$, and $f_\theta = \sum_j f_{\theta,j}$, which gives the target loss function of equation. 4

$$\text{minimize } L(\theta; \mathbf{f}_\theta) = \mathbb{E}_{u, \mathbf{a}} \left[\sum_{j=1}^K (f_{\theta,j}(u, \mathbf{a}) - r_j(u, \mathbf{a}))^2 \right] \quad (4)$$

Notice that until now we have made no proposition on the specification of the model architectures. There are whole bunches of neural network models proposed before to tackle different aspects of RS, including the well known Youtube RS([8]), Google deep-and-wide RS([6]). The model structure is highly coupled with the problem and the features. As we focus mainly on the intra-list correlation, in order to make the comparison easier, we propose several relatively simple, but representational models in the following part. We can surely add more complexity to those structures to account for more features and factors (such as dynamic user interest), but those are not referred in this paper.

We use $\phi(u)$ and $\phi(c_k)$ to denote the feature descriptor of user u and item c_k respectively, and $\phi(j)$ to represent the position embedding. In our experiments, the dimension of the feature descriptor is 24 for u , 32 for c_k and the embedding size is 8 for position j . The feature descriptor of the user and the item is a mixture of the embedding vector and other dense features. A sketch of the model can be found in fig. 2. Here we use the expression ‘‘Dense’’ to represent a linear mapping function with bias and activation function.

3.3.1 Multi Layer Perceptrons(MLP). The classic RS typically do not consider intra-list correlations at all, i.e. each item is evaluated independently based on the user information $\phi(u)$, item information

$\phi(c_{a_j})$, and position information $\phi(j)$ for considering the position bias. Our baseline model uses Multi-Layer Perceptrons(MLP) with the concatenation of the above three vectors as input(fig. 2a).

3.3.2 Gated Recurrent Neural Network(GRNN). We further propose to use GRNN([14]) to encode the preceding sequence $\mathbf{a}_j^-$ in order to capture the interactions between $\mathbf{a}_j^-$ and c_{a_j} , which is followed by two layer MLP(fig. 2b). Though applying single-direction recursive structures to inter-list correlations in RS has been reported before([16]), we report the first validation of GRNN in intra-list correlations. Compared with other diversified ranking algorithms, GRNN has the advantage that no additional assumption is introduced in the formulation of the correlation. However, GRNN has also deficiencies by presuming that $f_{\theta,j}$ is independent of $\mathbf{a}_j^+$. It assumes that the user exams all recommendations in top-bottom manner only, which is also the assumption of the cascade click model([7]). However, other studies have revealed that $\mathbf{a}_j^+$ does have impact in the overall performance of the j th position, including more sophisticated click models([32]) and mouse tracking studies([10]). Motivated by this, we propose to use Bi-GRNN([25]) and self-attention structures.

3.3.3 Bi-Directional Gated Recurrent Neural Network. To further improve the precision and take the following exposure $\mathbf{a}_j^+$ into account, we apply an second GRNN in reversed direction in addition to GRNN([25]), followed by similar MLP structures(fig. 2c)

3.3.4 Transformer. Initially proposed for neural machine translation, Transformer([30]) has achieved great successes as a sequence encoder. Self attention structure has been found to effectively capture the interactions intra-sequence. We firstly concatenate the user descriptor with item representation and position embedding, and then we apply a 2-layer Transformer to predict the probability of click in each position(fig. 2d).

3.3.5 Other Works.

While many previous works impose strong hypothesis on the formulations, Deep DPP ([33]) has effectively combined the essence of DPP and the representation power of Deep Learning to push the intra-list modeling to new frontier. DPP approximate the utility of the sequence $\mathbf{a}$ with $f_\theta(u, \mathbf{a}) \propto |\mathbf{K}_\theta(u, \mathbf{a})|$, where $\mathbf{K}_\theta = \{K_{\theta,ij}\}$ is a kernel matrix explicitly representing the correlation between each pair of position i, j , with $|\cdot|$ representing the determinant of the matrix. Given the clicked subset $\mathbf{a}_c = \{a_j | j \in [1, K], r_j = 1\}$, DPP tries to maximize $P_\theta(\mathbf{a}_c)$, defined in equation. 5

$$\text{maximize } P(\mathbf{a}_c) = \frac{|\mathbf{K}_\theta(u, \mathbf{a}_c)|}{|\mathbf{K}_\theta(u, \mathbf{a}) + \mathbf{I}|} \quad (5)$$

As proposed by Wilhelm et al, a neural mapping function can be used to map $u, \mathbf{a}$ to the kernel matrix([33]). In this paper we use the model architecture shown in equation. 6, where $D_{i,j}$ are the Jaccard distances between item descriptor c_i and item c_j , and α and σ are hyper-parameters.

$$\begin{aligned} e(u, k) &= \text{Dense}([\phi(u), \phi(c_k)]), \\ K_{\theta,ij}(u, \mathbf{a}) &= \alpha e(u, a_i) e(u, a_j) \exp\left(-\frac{D_{a_i, a_j}}{2\sigma^2}\right), \end{aligned} \quad (6)$$

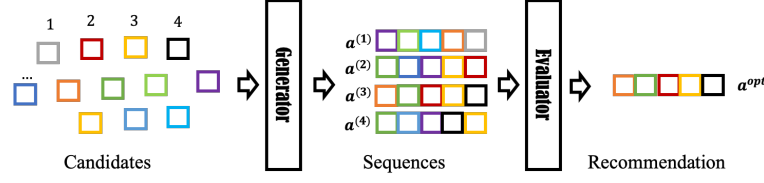


Figure 1: A sketch of the Evaluation-Generation framework. The Generator generates potential high-quality sequences with heuristic searching or reinforcement learning, from which the Evaluator further select the superior one.

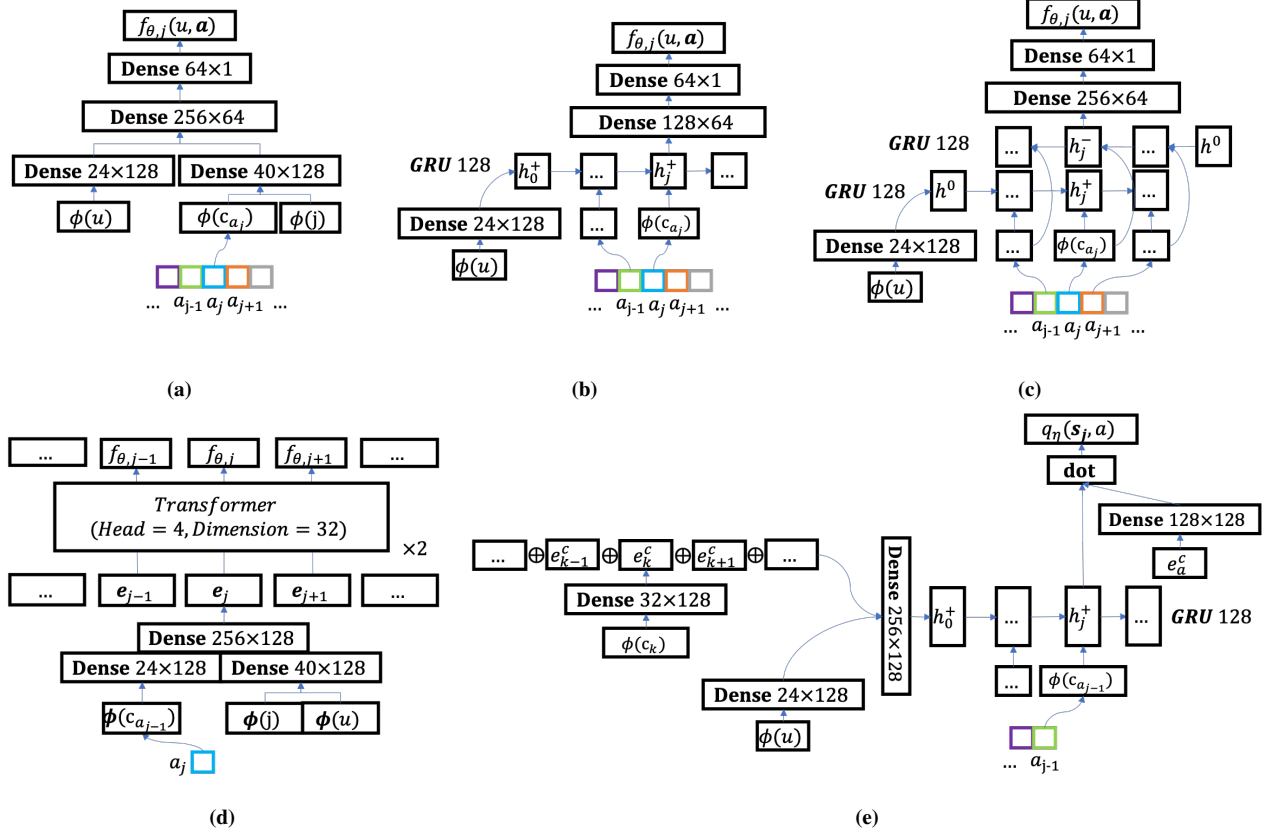


Figure 2: Sketches of the proposed model architectures. (a). Multi-Perceptron Layers (MLP) as the Evaluator or the Generator; (b). Gated Recurrent Neural Network (GRNN) as the Evaluator or the Generator; (c). Bi-Directional Gated Recurrent Neural Network (Bi-GRNN) as the Evaluator; (d). Transformer as the Evaluator; (e). SetToSeq as the Generator

3.4 The Generators

There exists various paradigm to generate an ordered sequence, such as deconvolution neural network ([37]) and sequential generations. In our case we choose the latter one, where the probability of generating the sequence $\mathbf{a}$, written as $\Pi_{\eta}(\mathbf{a}; u, \mathbf{c})$, is the product of sequential decisions π_{η} shown in Eq. 7.

$$\Pi_{\eta}(\mathbf{a}; u, \mathbf{C}) = \prod_{j=1}^K \pi_{\eta}(a_j; \mathbf{s}_j) \quad (7)$$

Here we denote the *state* as $\mathbf{s}_j = [u, \mathbf{a}_j^-, \mathbf{c}]$, the choice of the item index a_j is the *action* at step j . We use heuristic values to guide the policy $\pi_{\eta}(a_j; \mathbf{s}_j)$, which we also call *priority score*, denoted as $q_{\eta}(\mathbf{s}_j, a)$. It represents the *Value* of choosing action a at the state $\mathbf{s}_j$. With the priority score we apply either greedy policy (equation. 8) or categorical sampling policy (equation. 9, with τ being the hyper-parameter of temperature).

$$\pi_{\eta}(a; \mathbf{s}_j) = \begin{cases} 1.0 & \text{if } a = \operatorname{argmax}_{a'} q_{\eta}(\mathbf{s}_j, a') \\ 0.0 & \text{else} \end{cases} \quad (8)$$

$$\pi_\eta(a; \mathbf{s}_j) = \frac{\exp(q_\eta(\mathbf{s}_j, a)/\tau)}{\sum_{k=1}^N \exp(q_\eta(\mathbf{s}_j, k)/\tau)} \quad (9)$$

In order to effectively search for sequences that satisfies the target of equation. 2, we can use supervised learning or reinforcement learning for the priority score q_η . In the following part we introduce different learning metrics and model structures for q_η .

3.4.1 Supervised Learning. Predicting the utility $\mathbf{r}(u, \mathbf{a})$ directly by learning from the user feedback are relatively simple and straight-forward for q_η , which means we use the immediate feedback to approximate the priority score. Here the target function is shown in equation. 10. We use only u and $\mathbf{a}_j^-$ as the representation of state $\mathbf{s}_j$, but we drop the information of $\mathbf{c}$, as it is nearly of no help if we want to approximate the *immediate* reward only. The MLP(which is similar to fig. 2a) and GRNN(fig. 2b) are reasonable choices for q_η here, which do not rely on $\mathbf{a}^+$.

$$\text{minimize } L(\eta; \mathbf{q}_\eta) = \mathbb{E}_{u, \mathbf{a}} \left[\sum_j (q_\eta(\mathbf{s}_j, a_j) - r_j(u, \mathbf{a}))^2 \right] \quad (10)$$

3.4.2 Reinforcement Learning. Applying RL enables the the sequential decision to pursue long term reward in each step. We argue that the choice of model architecture for RL needs to account for candidates $\mathbf{c}$ in order to optimize the long term rewards. Here we propose a neural structure called the set to sequence decoder(SetToSeq for short).

SetToSeq borrows ideas from pointer-net([31]), which has been further combined with RL to solve universal combinatorial optimization(CO) problems([3]). The main difference between SetToSeq and pointer-net lies in that exchangeable operations(we use summation) are applied to keep the model order insensitive to the candidate set $\mathbf{c}$ in SetToSeq, while GRNN([14]) or LSTM([17]) is used to encode the candidates in pointer-net. Besides, we also inject the user and context information $\phi(u)$ in the generative layer. The detailed model structure can be found in fig. 2e.

Q Learning ([26]) with replay memory is an efficient and widely used off-policy RL algorithm, where the value function $q_\eta(a, \mathbf{s}_j)$ is to approximate the long term reward starting from the position j . As we are facing a finite horizon problem with static horizon(K), we use $\gamma = 1.0$ as the decay factor. The temporal difference error is written as equation. 11, where η' is a delayed copy of parameter η , typically known as target network.

$$L_{TD}(\eta, q_\eta) = \mathbb{E}_{u, \mathbf{a}} \left[(\max_a \{q_{\eta'}(a, \mathbf{s}_j)\} + f_{\theta, j}(u, \mathbf{a}) - q_\eta(a_j, \mathbf{s}_j))^2 \right] \quad (11)$$

Here we use the *simulated feedback* $f_{\theta, j}(u, \mathbf{a})$ instead of the the *real feedback* r_j as the reward, which is directly derived from equation. 2. However, it is also possible to use the real feedback r_j . We put some comments on the two kinds of feedback. Learning from simulated feedback takes risks as if the Generator "attacks" the Evaluator. In case discrepancies exist between the Evaluator and the realistic environment, the Generator would deviate from the real target. On the other hand, learning from real feedback in a realistic system typically requires quite heavy engineering work, or else it would result in off-policy learning with static data. Studies have

revealed that off-policy learning with static data sometimes is not guaranteed to work well enough([13]).

4 EXPERIMENTS

In our experiments, we use 100 million lists from user-system interaction records for training, and 1 million lists for testing, which were collected from Baidu App News Feed System (BANFS), one of the largest RS in China. BANFS has over hundreds of millions of daily active users. A sequence of 10 ~ 50 items are refreshed corresponding to the user requirement. In our experiment, to reduce the cost of the experiment, our offline dataset¹ contains only a subset of the features, including the user id, item id, item category and layout (the appearance of the item). In our experiment settings, we focus mainly on the clicks of the recommendation, again extension to multi-targets are straight-forward but not mentioned here.

4.1 Evaluation Metrics

Traditional IR evaluation metrics(such as NDCG, MAP, ERR) are not suitable as those are based on static data. Previous work toward evaluating combinatorial RS include Yue et al. using an artificial simulator ([35]), others using online experiments for evaluation ([33]). Also there are some counterfactual evaluation tricks ([18], [24]), but applying those metrics to realistic RS with over millions of candidate items and users is often intractable.

In this work, we evaluate our ideas from the following criterions.

- Firstly, the precision of the Evaluators are evaluated by three metrics with realistic user feedback. Area Under Curve(AUC) of the ROC curve is used to evaluate the precision of predicting the utility of each position in the sequence. **SeqRMSE** and **SeqCorr** are used to evaluate the precision of predicting the overall utility. **Sequence Root Mean Square Error (SeqRMSE)** is defined as

$$\text{SeqRMSE} = \sqrt{\mathbb{E}_{u, \mathbf{a}} [(f_\theta(u, \mathbf{a}) - r(u, \mathbf{a}))^2]}. \quad (12)$$

Since some methods, such as DPP, do not predict $\mathbb{E}[r(u, \mathbf{a})]$ explicitly, we also evaluate the correlation between the overall utility $r(u, \mathbf{a})$ and the evaluation score $f_\theta(u, \mathbf{a})$ (**SeqCorr** for short), which is defined in Eq. 13.

$$\text{SeqCorr} = \frac{\text{Cov}(f_\theta(u, \mathbf{a}), r(u, \mathbf{a}))}{\sqrt{\text{Var}(f_\theta(u, \mathbf{a})) \cdot \text{Var}(r(u, \mathbf{a}))}}. \quad (13)$$

- Secondly, we compare different Generators by regarding the Evaluator itself as an simulator(or environment). Previous work has proposed using simulators to evaluate combinatorial recommendation([35]). Building simulators to evaluate RL recommender systems offline has also been reported([29]). In our case, we argue that the Evaluator can work as both an **Selector** online and a natural **Simulator** offline. We do not only provide the comparison of different Generators under the proposed simulator, but we also demonstrate the validity of the simulator itself by directly comparing the Evaluator to the online environments.

¹Our code is released at <https://github.com/LihangLiu/Generator-Evaluator>, which is based on PaddlePaddle. Desensitized dataset will come soon.

- Finally, we publish the result to compare different ranking frameworks in online A/B tests. As we are more cautious toward online experiments, we did not carry out the experiments on all possible ranking frameworks. It is worth noticing that our online experiments uses larger feature set and datasets to achieve better performance, thus the performance of the online and offline experiments are not totally comparable.

4.2 Results on The Evaluators

To evaluate the precision of the Evaluator models, we use AUC, SeqRMSE and SeqCorr as the criteria for comparison. We compare the MLP, GRNN, Bi-GRNN and the Transformers, the architectures of which are shown from fig. 2a to fig. 2d. We use adam optimizer([20]) with the batch size of 768 and the hyper-parameters of learning rate = $1.0e-3$, $\beta_1 = 0.9$, $\beta_2 = 0.999$. For each model the optimizer goes through the training data for 20 epochs, and the last epoch is used.

By concluding from Tab. 1, we can see that $\mathbf{a}_j^-$ and $\mathbf{a}_j^+$ do make impact on the click of the j -th position, as the Bi-GRNN and the Transformer outperforms the other models in all three evaluation criteria. The performance of DPP is below the baseline, which is mainly caused by missing the position bias, while in BANFS the clicks are severely influenced by its position

Algorithm	AUC	SeqRMSE	SeqCorr
MLP	0.7706	0.4478	0.4733
GRNN	0.7754	0.4443	0.4853
Bi-GRNN	0.7798	0.4420	0.4936
Transformer	0.7794	0.4436	0.4906
Deep DPP	-	-	0.3810

Table 1: Offline comparison of different Evaluators

To visualize the intra-list correlation in BANFS, we plot the heat-map of the average attention strength on the self-attention layer of first layer in the Transformer(Fig. 3b). We mention several phenomena that are coherent with our instinct or investigation: Firstly, each item is more dependent on its previous items, especially those on the top of the list. The BANFS distributes 10 ~ 50 items as a sequence at each time, but only 2 ~ 4 items can be displayed in a screen(Fig. 3a). A user needs to slide downward to examine the whole list. Thus, whether the items on the top of the sequence attracts the user has an impact on the probability that the items lie below are examined, and thus clicked. Secondly, the attention between adjacent positions j and $j + 1$ is not as strong as that between j and $j + 2$, which makes the heat-map interweaving(like chessboard). To better study this phenomenon, we further plot the realistic correlation of clicks between each position pair. Fig. 3c shows that the correlation of user clicks is interweaving: the adjacent positions is less likely to be clicked together, but j and $j + 2$ is more likely to be clicked together. This is in consistency with the **Serpentining** phenomenon that was mentioned in [36]. This phenomenon has further shown that the intra-list correlation is much more complicated than many position bias hypothesis or unordered set-wise hypothesis previously proposed.

4.3 Results on the Generators

To evaluate the Generators, we randomly sample 1 million candidate sets from the user interaction logs. Those are regarded as pools of candidate $\mathbf{c}$. We begin a simulation where we randomly sample an user u , a candidate set $\mathbf{c}$ and length of the final list K in each step. The length K follows the real distribution of sequence length online, which varies between 10 and 50. We sample the candidate set such that $N = 2K$. Then, the Generator is required to generate one or multiple sequences of length K , and we use the Evaluator to pick the sequence with highest overall score (if only 1 list is generated, such as greedy picking, then there are no need of using Evaluators as a selector). We compare the statistical evaluation score $\mathbb{E}_{u, \mathbf{a} \sim \Pi_\eta} [f_\theta(u, \mathbf{a})]$ on the generated lists. For the training settings, in SL we keep the hyper-parameters the same as that of the Evaluator; in RL we use a replay memory of size 768K, and we keep the training batch numbers and learning rates the same as SL.

We choose three different Evaluators as the simulator, including GRNN, Bi-GRNN and Transformer. We use three different model architectures for the Generators(MLP, GRNN and SetToSeq), which is combined with different learning metrics(SL and RL) and policies(Greedy and Sampling). Notice that in case we use GRNN + SL as the Evaluator and the Generator at the same time, the sampling policy can also be replaced with beam search. We show the results in Tab. 2. Several remarks can be made, the comparison among different model architectures shows the importance of incorporating contexts $\mathbf{a}_j^-$ and candidates $\mathbf{c}$. The comparison of greedy policy and sampling policy shows that there is indeed non-negligible gap between local and global optimum. The RL group outperforms SL group under comparable policy, showing that RL can be used to improve the efficiency of searching for global optimum, or to reduce the number of the sampled trajectories which is required to achieve comparable performance.

To illustrate that the proposed framework indeed yields better item combinations, we did some inspection in the generated lists. The combination of item ids or high dimensional features are far too sparse to give any insightful results, thus we focus on the layouts of the short local segments. The layout marks the visual content arrangement within each item when shown to the users, as shown in Fig. 3a. More concretely, we denote the layout of the i -th item as $l_i \in [1, M]$, where M is the size of all distinct layouts. Then we consider the layouts of a segment of three consecutive items in position $(j, j + 1, j + 2)$ as the local pattern $P_{l_{sj}, l_{sj+1}, l_{sj+2}}$ that we want to look into. In BANFS, there are $M = 6$ types of layouts, e.g. "text-only", "one-image", "three-images", etc. Thus there are $6^3 = 216$ distinct layout patterns in total. For a recommendation list of length K , we count all possible $K - 2$ segments. The procedure of analyzing local patterns works as follows: Firstly, the average sum of click $\mathbb{E}[r(P_{l_{sj}, l_{sj+1}, l_{sj+2}})]$ of the segment pattern can be counted from the user log. Under the assumption that the higher click rate means better quality of the pattern, we use $\mathbb{E}[r(P_{l_i, l_j, l_k})]$ to measure the quality of the local layout pattern P_{l_i, l_j, l_k} . So we regard the layout patterns in 216 possible patterns that rank top-N in the expected clicks as "good patterns". To evaluate our proposed framework, we calculate the ratio of the top-N pattern segments in the generated lists from different Generators. We list the distribution on Top-K patterns with different ranking frameworks in fig. 4. The figure shows that the list

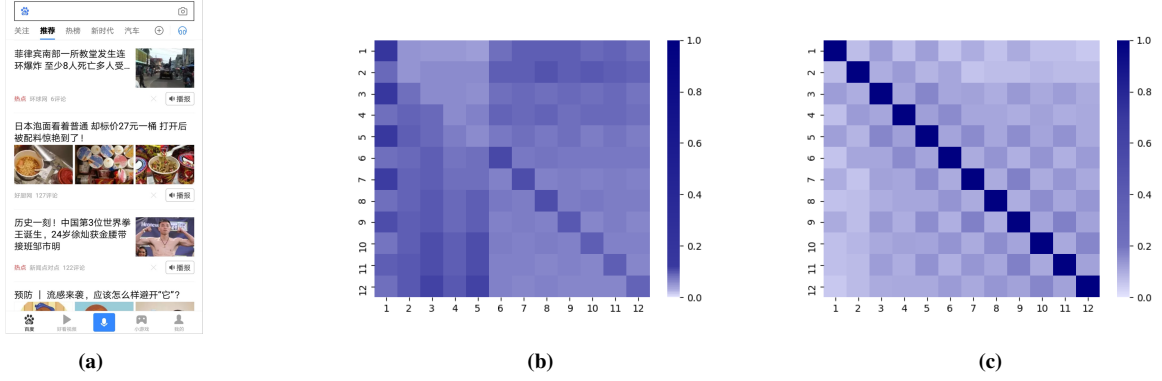


Figure 3: (a) A snapshot of BANFS. (b) Average attention weights between any two positions(x and y) over 6000 lists. (c) Correlation of realistic user clicks between any two positions, statistics from the same lists in user log.

Generator	Evaluator					
	Bi-GRNN			Transformer		
	Greedy	Sampling($n = 20$)	Sampling($n = 40$)	Greedy	Sampling($n = 20$)	Sampling($n = 40$)
MLP + SL	1.3538	1.7030	1.7540	1.3615	1.6290	1.6779
GRNN + SL	1.6652	1.8852	1.9226	1.5522	1.7853	1.8267
GRNN + RL(Simulated Data)	1.7552	1.9399	1.9683	1.8296	2.0239	2.0550
SetToSeq + RL(Simulated Data)	1.9174	2.0313	2.0606	1.9751	2.1000	2.1348
SetToSeq + RL(Real Data)	1.3449	1.5851	1.6259	-	-	-

Table 2: Offline comparison of different Generators by using the Evaluators for simulation.

with higher evaluation score indeed include more "good patterns", which also means the evaluation score is consistent with intuitive indicators.

4.4 Performance Online

Correlation between Evaluators and Online-Performance The previous results show that the Evaluator is more correlated to the sum of clicks of a list. But, is the predicted sum of clicks related to the final performance? Is it appropriate to treat Evaluator as a simulator? We perform additional investigation on the correlation between the Evaluation score of lists $f_\theta(u, \mathbf{a})$ and the performance of A/B test. Typically we judge whether a new policy is superior than the baseline, by the increase in some critical metrics, such as total user clicks, in A/B test. For two experiment groups with experiment ID I_A (experimental) and I_B (baseline), the **Relative Gain** in total clicks is defined as the relative increase of clicks compared with baseline. Thus we retrieve the logs of the past experiments, and we re-predict click of each sequences in the record by inferencing with our Evaluator model. We calculate the **Predicted Relative Gain** by

$$\Delta = \frac{\sum_{i \in I_A} f_\theta(u, \mathbf{a}_i) - \sum_{i \in I_B} f_\theta(u, \mathbf{a}_i)}{\sum_{i \in I_B} f_\theta(u, \mathbf{a}_i)}. \quad (14)$$

We collect over 400 A/B testing experiments during 2018, including not only new ranking strategies with various policy, model and new features, but also human rules. Some of the new strategies are tested to be positive, others negative. We counted the correlation between the predicted relative gain and the statistical real relative gain. We use MLP and Bi-GRNN Evaluator for comparison. The

correlation between MLP and online performance among the 400 experiments is 0.2282, while the correlation between Bi-GRNN and real performance is as high as 0.9680. It has proved to some extent that the Evaluator can evaluate a strategy before doing A/B test online and the confidence is relatively high, thus the simulation results by treating the Evaluator as the simulator are relatively confident.

	MLP	Bi-GRNN
Correlation with Online Performance	0.2282	0.9680

Table 3: Correlation between the Evaluator predictions and the online A/B tests

Online A/B Test of Ranking Framework In order to finally validate the proposed framework, we have conducted a series of online experiments on BANFS, we report the comparison of the following methods.

- *MLP*: Generator only, MLP + SL (Greedy)
- *GRNN*: Generator only, GRNN + SL (Greedy)
- *Evaluator-Generator*: GRNN + SL + Sampling($n = 20$) as the Generator, Bi-GRNN as the Selector.
- *SetToSeq + RL + Simulated Data*: Training the SetToSeq model with Q-Learning and simulated feedback. The simulated feedback comes from interacting with the Evaluator of Bi-GRNN.

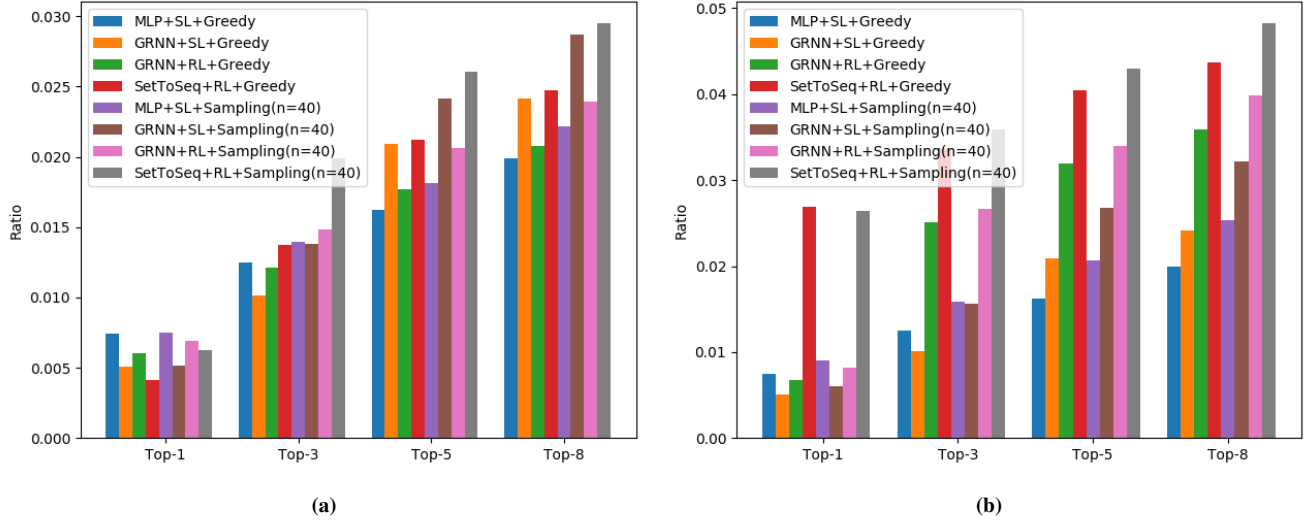


Figure 4: The ratio of Top-K patterns to all generated patterns($P_{I_{s_j}, I_{s_{j+1}}, I_{s_{j+2}}}$) with different Evaluation-Generation frameworks. (a) Bi-GRN as the Evaluator. (b) Transformer as the Evaluator

- *SetToSeq + RL + Real Data*: Training the SetToSeq model with Q-Learning and realistic clicks, in completely off-policy manner with finite log data.

Our evaluation result until now shows that Evaluator-Generator with softmax sampling Generator has state-of-the-art online performance. The RL group has shown comparable performance with Evaluator-Generator. Though we believe that RL should be more cost-efficient and more straightforward for solving the intra-list correlation, our experiments shows that the performance of RL occasionally generate unexpected bad patterns. We have also compared the Coverage([40]) on distinct categories(There are 40+ categories in all). It is verified that the proposed framework indeed improves diversity of exposure even though diversity is never considered as a explicit target in our framework(tab. 4).

5 DISCUSSIONS

In this paper, we propose a recommender framework by optimizing K-item in the result page as a whole. We propose the Evaluation-Generation framework to solve the combinatorial optimization problem. We show that compared with traditional diversified ranking algorithms, the proposed framework is capable of capturing various possible correlations as well as the position bias. In this section, we post some of our further comments in this framework and its possible future extensions.

Robustness of RL. Though Q-learning greatly outperformed the other learning metrics in off-line Evaluations, it is found that Q-learning is vulnerable to the noisy online policy. E.g., when some positions in the sequence are disturbed by the other mandatory interventions(which is normal in online system), the model can generate poor combinations. The robustness of RL in our case deserves further investigation.

Exploration and Exploitation. Exploration is important for interactive systems, as continuously greedy recommendation would

end up in mediocre or outdated contents. We propose that RS should also explore different combination of items besides the item itself. Suppose that the model see only "good" combinations, the system would not be able to learn to avoid "bad" ones such as duplicated recommendations. Thus we keep a small fraction of PV for exploring different combinations randomly online.

Synthesising Intra-list and Inter-list Relations. Typical RS has both the features of intra-list correlation and inter-list evolution. However, building unified framework to address both factors in realistic RS remains challenging. We believe it's a promising direction toward the next generation RS.

REFERENCES

- [1] Gediminas Adomavicius and Alexander Tuzhilin. Context-aware recommender systems. In *Recommender systems handbook*, pages 191–226. Springer, 2015.
- [2] Yossi Azar and Iftah Gamzu. Ranking with submodular valuations. In *Proceedings of the twenty-second annual ACM-SIAM symposium on Discrete Algorithms*, pages 1070–1079. SIAM, 2011.
- [3] Irwan Bello, Hieu Pham, Quoc V. Le, Mohammad Norouzi, and Samy Bengio. Neural combinatorial optimization with reinforcement learning. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Workshop Track Proceedings* [3].
- [4] Pedro G Campos, Fernando Díez, and Iván Cantador. Time-aware recommender systems: a comprehensive survey and analysis of existing evaluation protocols. *User Modeling and User-Adapted Interaction*, 24(1-2):67–119, 2014.
- [5] Olivier Chapelle and Ya Zhang. A dynamic bayesian network click model for web search ranking. In *Proceedings of the 18th international conference on World wide web*, pages 1–10. ACM, 2009.
- [6] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishu Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, et al. Wide & deep learning for recommender systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*, pages 7–10. ACM, 2016.
- [7] Aleksandr Chuklin, Ilya Markov, and Maarten de Rijke. Click models for web search. *Synthesis Lectures on Information Concepts, Retrieval, and Services*, 7(3):1–115, 2015.
- [8] Paul Covington, Jay Adams, and Emre Sargin. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems*, pages 191–198. ACM, 2016.
- [9] Nick Craswell, Onno Zoeter, Michael Taylor, and Bill Ramsey. An experimental comparison of click position-bias models. In *Proceedings of the 2008 international conference on web search and data mining*, pages 87–94. ACM, 2008.

Recommender Framework	Relative Gain (%)	Coverage of Categories(%)
GRNN vs. MLP	+1.75	+4.05
Evaluator-Generator vs. GRNN	+2.44	+0.62
SetToSeq + RL + Simulated Data vs. Evaluator-Generator	-0.01	-
SetToSeq + RL + Real Data vs. Evaluator-Generator	-0.05	-

Table 4: Comparison of different solutions in online performance

- [10] Fernando Diaz, Ryen White, Georg Buscher, and Dan Liebling. Robust models of mouse movement on dynamic web search results pages. In *Proceedings of the 22nd ACM international conference on Conference on information & knowledge management*, pages 1451–1460. ACM, 2013.
- [11] Bradley B Doll, Dylan A Simon, and Nathaniel D Daw. The ubiquity of model-based reinforcement learning. *Current opinion in neurobiology*, 22(6):1075–1081, 2012.
- [12] Yue Feng, Jun Xu, Yanyan Lan, Jiafeng Guo, Wei Zeng, and Xueqi Cheng. From greedy selection to exploratory decision-making: Diverse ranking with policy-value networks. In *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval, SIGIR 2018, Ann Arbor, MI, USA, July 08-12, 2018* [12], pages 125–134.
- [13] Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. *CoRR*, abs/1812.02900, 2018.
- [14] Alex Graves, Abdel Rahman Mohamed, and Geoffrey Hinton. Speech recognition with deep recurrent neural networks. In *IEEE International Conference on Acoustics*, 2013.
- [15] Xiangnan He, Hanwang Zhang, Min-Yen Kan, and Tat-Seng Chua. Fast matrix factorization for online recommendation with implicit feedback. In *Proceedings of the 39th International ACM SIGIR conference on Research and Development in Information Retrieval*, pages 549–558. ACM, 2016.
- [16] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. Session-based recommendations with recurrent neural networks. In *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings* [16].
- [17] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [18] Nan Jiang and Lihong Li. Doubly robust off-policy value evaluation for reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016* [18], pages 652–661.
- [19] Elias B. Khalil, Hanjun Dai, Yuyu Zhang, Bistra Dilkina, and Le Song. Learning combinatorial optimization algorithms over graphs. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, 4-9 December 2017, Long Beach, CA, USA*, pages 6351–6361, 2017.
- [20] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings* [20].
- [21] Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8), 2009.
- [22] Alex Kulesza and Ben Taskar. Determinantal point processes for machine learning. *CoRR*, abs/1207.6083, 2012.
- [23] Alex Kulesza, Ben Taskar, et al. Determinantal point processes for machine learning. *Foundations and Trends® in Machine Learning*, 5(2–3):123–286, 2012.
- [24] Lihong Li, Shunbao Chen, Jim Kleban, and Ankur Gupta. Counterfactual estimation and optimization of click metrics in search engines: A case study. In *Proceedings of the 24th International Conference on World Wide Web*, pages 929–934. ACM, 2015.
- [25] Xuezhe Ma and Eduard H. Hovy. End-to-end sequence labeling via bi-directional lstm-cnns-crf. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, ACL 2016, August 7-12, 2016, Berlin, Germany, Volume 1: Long Papers* [25].
- [26] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin A. Riedmiller. Playing atari with deep reinforcement learning. *CoRR*, abs/1312.5602, 2013.
- [27] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th international conference on World Wide Web*, pages 285–295. ACM, 2001.
- [28] Guy Shani and Asela Gunawardana. Evaluating recommendation systems. In *Recommender systems handbook*, pages 257–297. Springer, 2011.
- [29] Jing-Cheng Shi, Yang Yu, Qing Da, Shi-Yong Chen, and Anxiang Zeng. Virtual-taobao: Virtualizing real-world online retail environment for reinforcement learning. *CoRR*, abs/1805.10000, 2018.
- [30] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, pages 5998–6008, 2017.
- [31] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Advances in Neural Information Processing Systems*, pages 2692–2700, 2015.
- [32] Chao Wang, Yiqun Liu, Meng Wang, Ke Zhou, Jian-yun Nie, and Shaoping Ma. Incorporating non-sequential behavior into click models. In *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 283–292. ACM, 2015.
- [33] Mark Wilhelm, Ajith Ramanathan, Alexander Bonomo, Sagar Jain, Ed H Chi, and Jennifer Gillenwater. Practical diversified recommendations on youtube with determinantal point processes. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*, pages 2165–2173. ACM, 2018.
- [34] Yan Yan, Gaowen Liu, Sen Wang, Jian Zhang, and Kai Zheng. Graph-based clustering and ranking for diversified image search. *Multimedia Systems*, 23(1):41–52, 2017.
- [35] Yisong Yue and Carlos Guestrin. Linear submodular bandits and their application to diversified retrieval. In *Advances in Neural Information Processing Systems*, pages 2483–2491, 2011.
- [36] Qian Zhao, Gediminas Adomavicius, F Maxwell Harper, Martijn Willemsen, and Joseph A Konstan. Toward better interactions in recommender systems: cycling and serpentine approaches for top-n item lists. In *[CSCW2017] Proceedings of the 2017 ACM Conference on Computer Supported Cooperative Work and Social Computing*, 2017.
- [37] Xiangyu Zhao, Long Xia, Liang Zhang, Zhuoye Ding, Dawei Yin, and Jiliang Tang. Deep reinforcement learning for page-wise recommendations. In *Proceedings of the 12th ACM Conference on Recommender Systems, RecSys 2018, Vancouver, BC, Canada, October 2-7, 2018* [37], pages 95–103.
- [38] Guanjie Zheng, Fuzheng Zhang, Zihan Zheng, Yang Xiang, Nicholas Jing Yuan, Xing Xie, and Zhenhui Li. Drn: A deep reinforcement learning framework for news recommendation. In *Proceedings of the 2018 World Wide Web Conference on World Wide Web*, pages 167–176. International World Wide Web Conferences Steering Committee, 2018.
- [39] Xiaojin Zhu, Andrew Goldberg, Jurgen Van Gael, and David Andrzejewski. Improving diversity in ranking using absorbing random walks. In *Human Language Technologies 2007: The Conference of the North American Chapter of the Association for Computational Linguistics; Proceedings of the Main Conference*, pages 97–104, 2007.
- [40] Cai-Nicolas Ziegler, Sean M McNee, Joseph A Konstan, and Georg Lausen. Improving recommendation lists through topic diversification. In *Proceedings of the 14th international conference on World Wide Web*, pages 22–32. ACM, 2005.